

Algorithms and Probability

Course summary



Contents

1	Graph Theory	4
1.1	Connectivity	4
1.1.1	Cut Vertices and Cut Edges	5
1.1.2	Blocks	7
1.2	Cycles	8
1.2.1	Eulerian Cycles	8
1.2.2	Hamiltonian Cycles	8
1.2.3	Travelling Salesman Problem	9
1.3	Matchings	10
1.3.1	Hall's Marriage Theorem	12
1.4	Coloring	13
2	Probability Theory	15
2.1	Basics	15

2.2	Conditional Probability	17
2.3	Independence	18
2.4	Random Variables	18
2.5	Expected Value	19
2.6	Variance	20
2.7	Probability Distribution	21
2.7.1	Bernoulli Distribution	21
2.7.2	Binomial Distribution	21
2.7.3	Geometric Distribution	22
2.7.4	Negative Binomial Distribution	22
2.7.5	Poisson Distribution	22
2.8	Multiple Random Variables	23
2.8.1	Independence of Random Variables	23
2.9	Probability Estimations	23
2.10	Coupon Collector Problem	25
3	Randomized Algorithms	26
3.1	Error Reduction	26
3.2	Target Shooting	27
3.3	Prime Number Test	28
3.3.1	Euclid Certificate	28
3.3.2	Fermat Certificate	28
3.3.3	Miller-Rabin Certificate	29
3.4	Finding Duplicates	31
3.5	Bloom Filters	31
4	Algorithms Highlights	32
4.1	Networks and Flow	32
4.2	Augmenting Paths	33
4.2.1	Matchings in Bipartite Graphs	35
4.2.2	Vertex and Edge Disjoint Paths	36
4.3	Minimal Cuts in Graphs	36
4.4	Smallest Enclosing Circle	37
4.5	Convex Hull	39
4.5.1	Jarvis Wrap	40
4.5.2	Local Repair	41

This document is a summary of the Algorithms and Probability course that is lectured in the 2nd semester of Computer Science at ETH Zurich. I created this document to summarize and repeat the key learnings of this course. It may contain errors. Franz Schwinn, 08.2022.

1 Graph Theory

1.1 Connectivity

Definition

A graph $G = (V, E)$ is **connected** if

- $\forall u, v \in V, (u \neq v), \exists$ a u - v -path in G (a path connecting u with v).

In order to find out how strongly connected a graph is, we define k -connectivity and k -edge-connectivity.

Definition

An undirected graph $G = (V, E)$ is **k -connected** if

- $|V| \geq k + 1$ and
- $\forall X \subseteq V, X < k : G(V \setminus X, E)$ is connected

Definition

A graph $G = (V, E)$ is **k -edge-connected** if

- $\forall X \subseteq E, X < k : G(V, E \setminus X)$ is connected

Intuition: "At least how many vertices / edges do we need to remove from G such that G is not connected anymore?" Further we have:

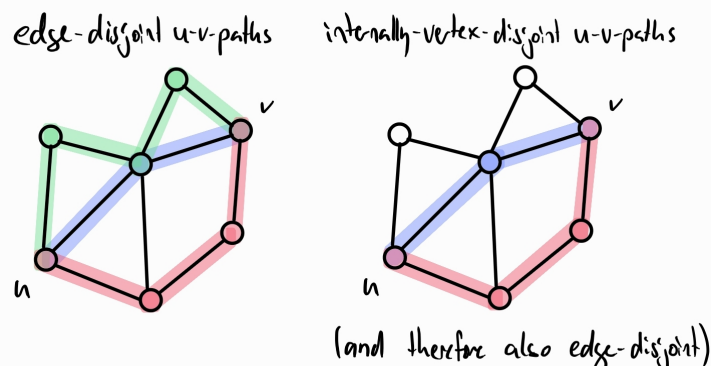
$$\text{connectivity} \leq \text{edge-connectivity} \leq \delta(G) \text{ (minimal degree among all vertices in } G\text{)}$$

An equivalent definition of connectivity is Menger's theorem:

Theorem

Menger's theorem: Let $G = (V, E)$ be a graph, then

- G is k -connected $\Leftrightarrow \forall u, v \in V, (u \neq v)$ there are at least k internally-vertex-disjoint u - v -paths
- G is k -edge-connected $\Leftrightarrow \forall u, v \in V, (u \neq v)$ there are at least k edge-disjoint u - v -paths



1.1.1 Cut Vertices and Cut Edges

Definition

Let $G = (V, E)$ be a connected graph. A vertex $v \in V$ is called a **cut vertex** if
► $G(V \setminus \{v\}, E)$ is not connected

Definition

Let $G = (V, E)$ be a connected graph. An edge $e \in E$ is called a **cut edge** if
► $G(V, E \setminus \{e\})$ is not connected

An important property of cut edges is the following.

Useful

Let $\{u, v\} \in E$ be an undirected cut edge. Then for both of its incident vertices u and v it holds that either $\text{deg}(u) = 1$ or u is a **cut vertex** (equivalently for v).

A modified version of the DFS allows us to find cut vertices and cut edges in a connected graph in $\mathcal{O}(|E|)$ time.

Algorithm 1: find_cut_vertices(G, n, root)

```
1 dfs = int[n];
2 low = int[n];
3 iscut = boolean[n];
4 int num = 0;
5 dfs(root);
6 if deg(root) ≥ 2 then
7   iscut[root] = true;
8 else
9   iscut[root] = false;
10 end
```

Algorithm 2: dfs(u)

```
1 num = num + 1;
2 dfs[u] = num;
3 low[u] = num;
4 iscut[u] = false;
5 for {u,v} ∈ E do
6   if dfs[v] == 0 then
7     int val = dfs(v);
8     if val ≥ dfs[u] then
9       iscut[u] = true;
10    end
11    low[u] = min(low[u], val);
12  else
13    low[u] = min(low[u], dfs[v]);
14  end
15 end
16 return low[u];
```

```

1 private ArrayList<Integer>[] g; // graph as adjacency list
2 private int[] dfs; // used to store dfs values
3 private int[] low; // used to store low values
4 private boolean[] iscut; // used to mark cut vertices
5 private int num; // marks the dfs value
6
7 public static boolean[] find_cut_vertices(ArrayList<Integer>[] g) {
8     this.g = g; // set graph
9     int n = g.length; // amount of vertices
10    dfs = new int[n]; // initialize dfs array
11    low = new int[n]; // initialize low array
12    iscut = new boolean[n]; // initialize iscut array
13    num = 0; // set dfs value to zero
14
15    dfs(0) // call modified dfs algorithm with vertex 0 as root
16    if(g[0].size() >= 2) { // if root has at least 2 children
17        iscut[0] = true;
18    } else {
19        iscut[0] = false;
20    }
21    return iscut;
22 }
23
24 private static int dfs(int u) {
25     n += 1; // increase dfs value
26     dfs[u] = n; // set dfs value
27     low[u] = n; // set low value
28     for(int i = 0; i < g[u].size(); i++) { // iterate through all adjacent
29         // vertices
30         int v = g[u].get(i); // get adjacent vertex
31         if(dfs[v] == 0) { // if we didn't touch vertex yet...
32             int val = dfs(v); // ...call dfs on it and store its low value
33             if(val >= dfs[u]) { // if there is no back edge...
34                 iscut[u] = true; // ...we know that u is a cut vertex
35             }
36             low[u] = Math.min(low[u], val); // compute low value of u
37         } else {
38             low[u] = Math.min(low[u], dfs[v]); // compute low value of u
39         }
40     }
41     return low[u];
42 }

```

As shown above, we modify the dfs algorithm by adding a *low* value to every vertex. $low[u]$ stores the smallest dfs value of all vertices that can be reached from u by using indefinitely many edges from the dfs tree but only one edge that is not part of the dfs tree.

Using the *low* value we can determine whether or not a given vertex is a cut vertex.

Useful

$v \in V$ is a cut vertex $\Leftrightarrow$

- v is the root of the dfs tree and $deg(v) \geq 2$ or
- v is not the root of the dfs tree and $\exists u \in V$ with $\{v, u\} \in E$ such that $low[v] \geq dfs[u]$

We can also use the *low* value to find cut edges.

Useful

- $\{u, v\} \in E$ is a cut edge $\Leftrightarrow$
- $\{u, v\}$ is part of the dfs tree and
 - $low[v] > dfs[u]$ (or $low[u] > dfs[v]$)

Since we require a connected graph as input, we know that $|V| \leq |E| - 1$ (thus $|E|$ is an upper bound of $|V|$) which means that the run time of our algorithm is simply $\mathcal{O}(|E|)$ instead of $\mathcal{O}(|E| + |V|)$ for default dfs.

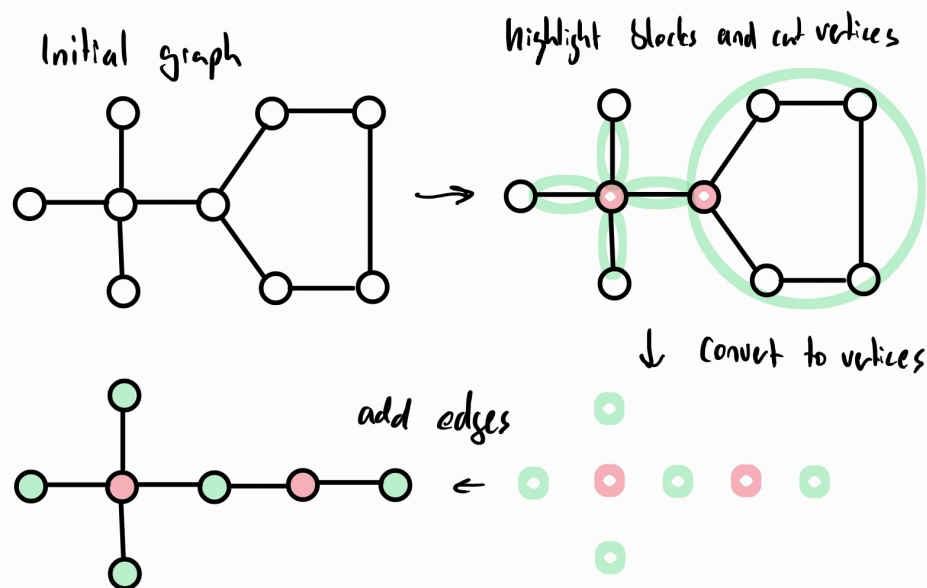
1.1.2 Blocks

Definition

A block is an equivalence relation between arbitrary edges $e, f \in E$ defined by:

$$e \sim f \Leftrightarrow \begin{cases} e = f \\ \exists \text{ a cycle that contains } e \text{ and } f \end{cases}$$

We can construct a block diagram from any given graph as shown below.



There are two useful pieces of information regarding blocks.

Useful

- A vertex is a cut vertex $\Leftrightarrow$ it is incident to at least two edges that are in different blocks.
- An edge is a cut edge $\Leftrightarrow$ it is the only edge in its block.

1.2 Cycles

1.2.1 Eulerian Cycles

Definition

An **Eulerian cycle** is a closed walk that traverses every edge of a given graph G exactly once.

We know that G is Eulerian if

- ▶ G is connected and
- ▶ $\forall v \in V : \deg(v) \% 2 == 0$ (all vertices have even degree)

If we know that G is Eulerian, we can find an Eulerian cycle in $\mathcal{O}(|E|)$ time. For this we use the "fast runner / slow runner" idea. First we pick an arbitrary vertex $v \in V$ and let the fast runner start from there. The runner chooses an arbitrary path W and immediately deletes every edge that he traversed. Therefore the runner can traverse every edge at most once. The runner stops as soon as a vertex with no outgoing edge (degree 0) is reached. This vertex must be v (the vertex that the runner started from). So the path W that the runner chose is a circuit.

Proof by contradiction: Let us assume that W ends in some vertex $u \in V \setminus \{v\}$ and that the runner already visited vertex u k times. Then on every of the k visits of u , the runner would have removed two edges incident to u (one edge on the way in to u and one edge on the way out from u). Furthermore, the runner would have removed one additional incident edge on his final way to u (last edge of the run). So in total, the runner would have removed $2k + 1$ edges that are incident to u , which means that an odd number of edges would have been removed. As we know that u had even degree in the beginning (requirement of G to be Eulerian), we can conclude that u now must have odd degree (since we subtracted an odd number of incident edges) and in particular u cannot have degree 0. This is a contradiction to our initial assumption that W must end in a vertex that has degree 0.

In case W contains all edges of G we successfully found an Eulerian cycle. If not, we know that all vertices in G still have even degree (because the runner always removed an even number of edges from a vertex that was visited). Next, the slow runner goes to search for a vertex v' in W with $\deg(v') > 0$. There must be such a v' , because if there was no such vertex we would know that G initially was not connected, which violates the requirements for G to be Eulerian. Once the slow runner found such a v' , we let the fast runner begin another run W' starting from v' . We then unite W and W' by creating a new circuit W_2 that contains all edges from W up until v' , followed by all edges from W' and then all remaining edges in W . We repeat this process until W_x contains all edges in G .

1.2.2 Hamiltonian Cycles

Definition

A **Hamiltonian cycle** is a cycle that visits every vertex of G exactly once.

As of today it is assumed that there is no algorithm with polynomial run time that is able to check if a graph is Hamiltonian. Therefore, determining whether a graph is Hamiltonian is NP hard. However, if a graph fulfills certain conditions we know that it must be Hamiltonian.

- ▶ Any $n \times m$ grid graph is Hamiltonian $\Leftrightarrow n * m$ is even
- ▶ Any d -dimensional hypercube ($V: 0, 1^d$, E : All pairs of vertices that differ in exactly one coordinate) are Hamiltonian

Theorem

Dirac's theorem: Let $G = (V, E)$ be a graph. G is Hamiltonian if

- ▶ $|V| \geq 3$ and
- ▶ $\delta(G) \geq \frac{|V|}{2}$

The reverse implication does not hold.

$$G \text{ is Hamiltonian} \not\Rightarrow |V| \geq 3 \text{ and } \delta(G) \geq \frac{|V|}{2}$$

Using a dynamic programming algorithm we can check if G is Hamiltonian in $\mathcal{O}(|V|^2 * 2^{|V|})$ time.

1.2.3 Travelling Salesman Problem

The Travelling Salesman Problem (TSP) is a well known problem:

Calculate the least expensive cycle on a complete graph K_n .

So we are looking for the minimal ("least expensive") Hamiltonian cycle in a given complete graph K_n . We call the optimal solution to this problem $opt(K_n, l)$ (the function opt takes in the graph K_n , the cost function l , and returns the minimal cost it requires to visit all vertices).

Definition

In the context of the TSP, we let a α -approximation algorithm be an algorithm that always comes up with a solution $C \subseteq E$ such that the following holds:

$$\sum_{e \in C} l(e) \leq \alpha * opt(K_n, l)$$

Or intuitively, an α -approximation algorithm always solves the TSP at least α times as well as the optimal solution. In spite of the fact that the TSP is NP hard, we can come up with efficient solutions if we assume that the triangle inequality ($l(\{x, z\}) \leq l(\{x, y\}) + l(\{y, z\}) \forall x, y, z \in V (x \neq y \neq z)$) holds.

When the triangle inequality holds, the TSP problem is called metric (metric TSP). There is a 2-approximation algorithm for the metric TSP with run time $\mathcal{O}(n^2)$.

1. We compute an Minimum Spanning Tree T in G .

$$l(T) \leq opt(K_n, l)$$

2. We create a multigraph by doubling every edge in T .

$$2 * l(T) \leq 2 * opt(K_n, l)$$

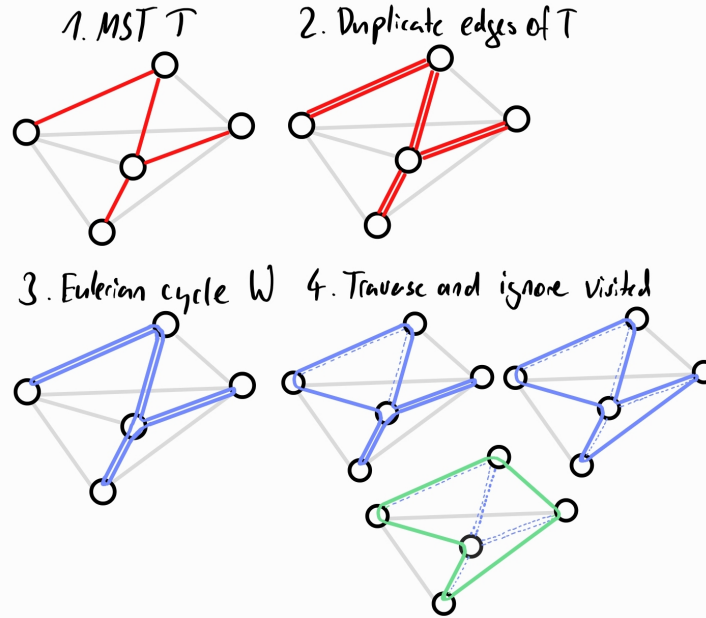
3. We find an Eulerian circuit W in T (which works because after doubling, all degrees of T are for sure even).

$$l(W) = 2 * l(T) \leq 2 * opt(K_n, l)$$

4. We now traverse the Eulerian circuit, but every time we encounter a vertex that we already visited, we skip it and directly move on with the next unvisited vertex in T . This is the point where we make use of the triangle inequality. We now have the Hamiltonian cycle C .

$$l(C) \leq l(W) = 2 * l(T) \leq 2 * \text{opt}(K_n, l)$$

The above statement shows that $l(C)$ is a 2-approximation.



There also is an $\frac{3}{2}$ -approximation algorithm for the metric TSP with $\mathcal{O}(n^3)$ run time.

1. Compute MST T in G .
2. Find minimal perfect matching M in G between vertices in T with odd degree. There is always an even number of vertices with odd degree, as this is a direct conclusion of the handshaking lemma:

$$\sum_{\forall v \in V} \deg(v) = 2 * |E|$$

3. Unite $T = T \cup M$.
4. Find Eulerian cycle S in T (S exists because all degrees are even).
5. Traverse S while skipping already visited vertices (making use of the triangle inequality, analogously to step 4 in the 2-approximation algorithm). This gives us the Hamiltonian cycle C .

1.3 Matchings

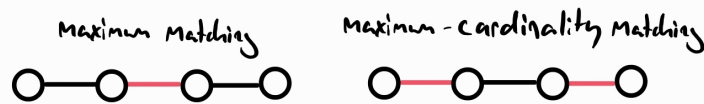
Definition

Let $G = (V, E)$ be a graph. A set of edges $M \subseteq E$ is called a **matching** if no vertex $v \in V$ is incident to more than one edge in M . Or formally:

$$\forall e, f \in M, (e \neq f) : e \cap f = \emptyset$$

We say that a vertex $v \in V$ is **matched** by M if $v \in e \in M$. We call a matching **perfect** if all vertices $v \in V$ in the graph are matched by M (or in other words: M is perfect $\Leftrightarrow |M| = \frac{|V|}{2}$).

What we often care about is to find a matching with the largest cardinality possible.



Definition

Let $G = (V, E)$ be a graph and M a matching in G .

- M is a **maximum-cardinality** matching if there is no M' such that $|M'| > |M|$
- M is a **maximum** matching if $\forall e \in E \setminus M : M \cup \{e\}$ is not a matching.

A maximum-cardinality matching is therefore also a maximum matching.

We can determine a maximum matching in $\mathcal{O}(|E|)$ by using the below greedy algorithm.

Algorithm 3: greedy_max(G)

```

1  $M = \emptyset$ 
2 while  $E \neq \emptyset$  do
3   pick  $e \in E$  randomly
4    $M = M \cup \{e\}$ 
5   delete  $e$  with all its incident edges from  $G$ 
6 end
7 return  $M$ 
```

Useful

The cardinality of M_{greedy} can be lower bounded as follows:

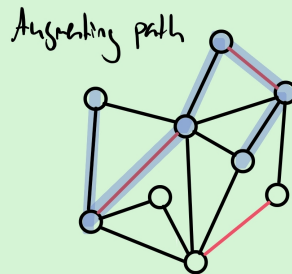
$$\frac{1}{2}|M_{\text{max-card}}| \leq |M_{\text{greedy}}| \leq |M_{\text{max-card}}|$$

Let us introduce augmenting paths.

Definition

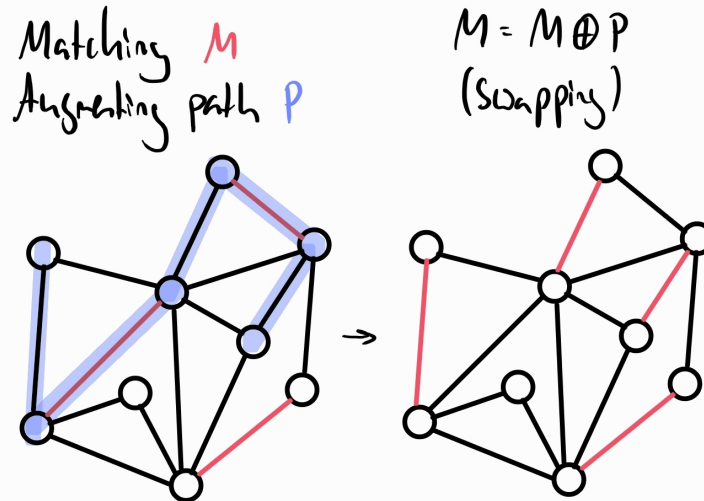
Let $G = (V, E)$ be a graph and M be a matching in G . A path P in G is called an **augmenting path** if

- its edges, alternately, are elements of M and $E \setminus M$ and
- its first vertex, last vertex, first edge, and last edge are unmatched by M



We can use augmenting paths to increase the size of a matching. Once we found an augmenting path P , we swap its edges by removing all covered edges in P from M and adding all uncovered edges in P to M . This results in M to increase in size by 1.

Using augmenting paths we can build an algorithm that finds a maximum-cardinality matching in a bipartite graph in $\mathcal{O}(|V||E|)$ time. We denote the swap operation as $M \oplus P$ (XOR operation, M is a matching and P an augmenting path).



We can find an augmenting path in a bipartite graph by using a modified version of BFS. Let $G = (A \uplus B, E)$ be a bipartite graph and M a matching in G . We add two vertices s and t to the graph G and draw a directed edge from s to all unmatched vertices in A and a directed edge from all unmatched vertices in B to t . Additionally, we set the direction of unmatched edges such that they point from A to B and matched edges such that they point from B to A . Then we run BFS on s and return the shortest path from s to t .

Algorithm 4: `find_augmenting_path( $G, M$ )`

- 1 add vertices s and t
 - 2 draw directed edge from s to all unmatched vertices in A
 - 3 draw directed edge from all unmatched vertices in B to t
 - 4 set direction of unmatched edges such that they point from $A \rightarrow B$
 - 5 set direction of matched edges such that they point from $B \rightarrow A$
 - 6 find shortest path P from s to t using BFS
 - 7 **return** $P \setminus \{s, t\}$
-

There are faster algorithms to find maximum-cardinality matchings.

Useful

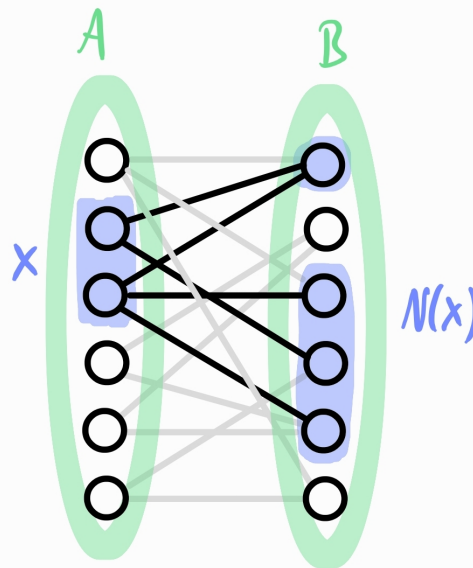
- A maximum-cardinality matching can be found in
- $\mathcal{O}(\sqrt{|V|}|E|)$ if the graph is bipartite using **Hopcroft-Karp** algorithm
 - $\mathcal{O}(|E||V|^2)$ in a general graph using **Blossom** algorithm

1.3.1 Hall's Marriage Theorem

Theorem

Hall's marriage theorem: Let $G = (A \uplus B, E)$ be a bipartite graph. G contains a matching of cardinality $|A| \Leftrightarrow \forall X \subseteq A : |X| \leq |N(X)|$.

Or intuitively, G contains a matching with cardinality $|A|$, if and only if for every subset of A , the neighborhood of that subset in B is larger than or equal to the size of the subset itself.



Definition

Frobenius' theorem: All k -regular bipartite graphs contain a perfect matching. It can be found in $\mathcal{O}(|E|)$ time.

For $k = 2^k$, we can use the below algorithm.

Algorithm 5: $2^k_max(G)$

```

1 while  $k \geq 2$  do
2   find Eulerian cycle  $T$  in  $G$ 
3   delete every second edge of  $T$  in  $G$ 
4 end
5 return  $G$ 
```

For a general k the procedure is harder and not discussed in this course.

1.4 Coloring

Definition

A **coloring** of a graph $G = (V, E)$ using k colors is a function $c : V \rightarrow \{1, \dots, k\}$ such that

$$\forall \{u, v\} \in E : c(u) \neq c(v)$$

The **chromatic number** $\chi(G)$ is the minimal number of colors it takes to color G .

Useful

Graphs with a chromatic number k can be called k -partite, since

$$\chi(G) \leq k \Leftrightarrow G \text{ is } k\text{-partite}$$

Finding the chromatic number of a graph is NP hard. However, we can use a greedy algorithm to color a graph in $\mathcal{O}(|E|)$ time.

Algorithm 6: greedy_coloring(G)

```
1 generate an arbitrary order of all vertices:  $\{v_1, \dots, v_n\}$ 
2  $c(v_1) = 1$ 
3 for  $i = 2, \dots, n$  do
4    $c(v_i) = \min(k \in \mathbb{N} | k \neq c(u) \forall u \in N(v_i) \cap \{v_1, \dots, v_{i-1}\})$ 
5 end
```

Useful

- For every order $\{v_1, \dots, v_n\}$ at most $\Delta(G) + 1$ colors are required to color G ($\Delta(G)$ is the maximum degree among all vertices in G).
- There always exists an order $\{v_1, \dots, v_n\}$ such that the greedy algorithm only needs $\chi(G)$ colors.
- Therefore we can bound the amount of colors the greedy algorithm needs ($C(G)$):

$$\chi(G) \leq C(G) \leq \Delta(G) + 1$$

Theorem

Brooks' theorem: Let $G = (V, E)$ be a connected undirected graph. If $G \neq K_n$ (complete graph) and $G \neq C_{2n+1}$ (cycle with odd number of vertices), then G can be colored in $\mathcal{O}(|E|)$ time using at most $\Delta(G)$ colors.

2 Probability Theory

2.1 Basics

Definition

A **probability space** consists of

- ▶ a sample space $\Omega = \{\omega_1, \dots, \omega_n\}$ containing elementary events $\omega_i \in \Omega$
- ▶ a probability function $Pr : \Omega \rightarrow [0, 1]$

The probability function measures the likelihood of an event to happen. The following two properties are required for the probability space to be correct:

- ▶ $\forall \omega_i \in \Omega : 0 \leq Pr[\omega_i] \leq 1$
- ▶ $\sum_{i=1}^n Pr[\omega_i] = 1$

A set $E \subseteq \Omega$ is called an event, and its probability is defined as the sum of the elementary events included in E .

$$Pr[E] = \sum_{\omega \in E} Pr[\omega]$$

As an example we can model the act of rolling a dice as a probability space. $\Omega = \{1, 2, 3, 4, 5, 6\}$ and $\forall \omega \in \Omega : Pr[\omega] = \frac{1}{6}$. We can define an event *roll an even number* as $E = \{2, 4, 6\}$.

Useful

For all events E we have

- ▶ $Pr[\emptyset] = 0$
- ▶ $Pr[\Omega] = 1$
- ▶ $0 \leq Pr[E] \leq 1$
- ▶ $Pr[\bar{E}] = 1 - Pr[E]$
- ▶ $E \subseteq F \Rightarrow Pr[E] \leq Pr[F]$

Theorem

Addition theorem: Let the events $A_1, \dots, A_n$ be pairwise disjoint (for all pairs $i \neq j$: $A_i \cap A_j = \emptyset$), then

$$Pr\left[\bigcup_{i=1}^n A_i\right] = \sum_{i=1}^n Pr[A_i]$$

This only holds for pairwise disjoint events. If we are not sure if the events are pairwise disjoint, we can use the following upper bound.

Theorem

Union Bound: For events $A_1, \dots, A_n$ we have

$$Pr\left[\bigcup_{i=1}^n A_i\right] \leq \sum_{i=1}^n Pr[A_i]$$

If we want to calculate the probability of a union of some events (they don't have to be pairwise disjoint), we can use the below procedure.

Theorem

Inclusion-Exclusion Principle: Let $A_1, \dots, A_n$ be some events. Then we have

$$Pr[\bigcup_{i=1}^n A_i] = \sum_{i=1}^n Pr[A_i] \quad (1)$$

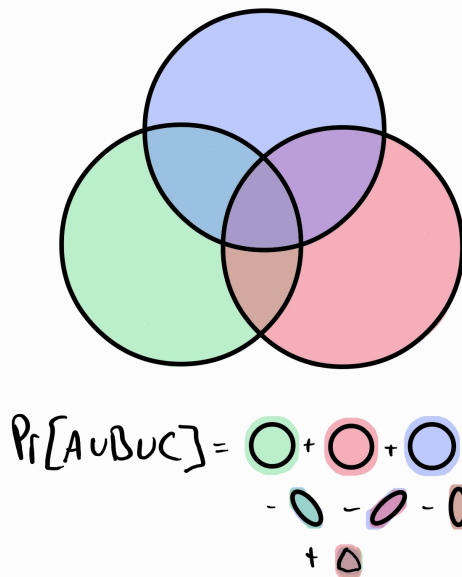
$$- \sum_{1 \leq i_1 \leq i_2 \leq n} Pr[A_{i_1} \cap A_{i_2}] \quad (2)$$

$$+ \sum_{1 \leq i_1 \leq i_2 \leq i_3 \leq n} Pr[A_{i_1} \cap A_{i_2} \cap A_{i_3}] \quad (3)$$

$$- \dots \quad (4)$$

$$+ (-1)^{n+1} * Pr[A_1 \cap \dots \cap A_n] \quad (5)$$

Or intuitively: If the number of events we intersect is odd, then we add the probability of the intersection and if the number of events we intersect is even, then we subtract the probability of the intersection.

**Definition**

Laplace's probability space: In case we don't know anything about the probability function Pr , we assume that all elementary events $\omega \in \Omega$ have the same probability to occur. We call such a probability space a Laplace space.

► $\forall \omega \in \Omega : Pr[\omega] = \frac{1}{|\Omega|}$

► for any event $E \subseteq \Omega : Pr[E] = \frac{|E|}{|\Omega|}$

Useful formulas for combinatorics:

Useful

Number of combinations:

- with repetition, ordered: n^k
- with repetition, unordered: $\binom{n+k-1}{k}$
- without repetition, ordered: $\frac{n!}{(n-k)!}$
- without repetition, unordered: $\binom{n}{k}$

Where n is the total number of elements (set size) and k is the number of elements drawn (subset size) and $\binom{n}{k} = \frac{n!}{(n-k)!k!}$.

2.2 Conditional Probability

By adding more information the probability for an event to occur may change.

Definition

Let A and B be events ($Pr[B] \geq 0$). The **conditional probability** $Pr[A|B]$ (A given B) is defined as:

$$Pr[A|B] = \frac{Pr[A \cap B]}{Pr[B]}$$

As an example we have the following problem: *Family X has two two children. What is the likelihood that both children are girls?* The probability space would be $\Omega = \{bb, bg, gb, gg\}$ (b short for *boy* and g short for *girl*). Without additional information we assume that $Pr[\omega] = \frac{1}{4}$. But the parents now tell us that (at least) one of the children is a girl. We can therefore perform the below calculation:

$$Pr[X = \text{"Both are girls"} | Y = \text{"One is girl"}] = \frac{Pr[X \cap Y]}{Pr[Y]} = \frac{Pr[\{gg\}]}{Pr[\{gg, gb, bg\}]} = \frac{\frac{1}{4}}{\frac{3}{4}} = \frac{1}{3}$$

We often use the following equality for conditional probability. It helps us to restructure problems in order to make them easier to calculate.

$$Pr[A \cap B] = Pr[A|B] * Pr[B]$$

Definition

Multiplication theorem: Let $A_1, \dots, A_n$ be events. If $Pr[A_1 \cap \dots \cap A_n] > 0$ then $Pr[A_1 \cap \dots \cap A_n] = Pr[A_1] * Pr[A_2|A_1] * Pr[A_3|A_1 \cap A_2] * \dots * Pr[A_n|A_1 \cap \dots \cap A_{n-1}]$

We can use this theorem to solve this problem: *What is the probability that at least two persons in a group of m people celebrate their birthday on the same day of the year?* We can rephrase this problem to another problem: *We randomly throw m balls in n baskets. What is the probability that there is no basket with more than one ball in the end?* We define $A_i = i$ -th ball lands in an empty basket. For the 1st throw it is clear that the ball will land in an empty basket, therefore $Pr[A_1] = 1$. For A_2 there are $n - 1$ baskets left, so $Pr[A_2|A_1] = \frac{n-1}{n}$. In general we have $Pr[A_i|A_1 \cap \dots \cap A_{i-1}] = \frac{n-(i-1)}{n}$. Now we apply the multiplication theorem and get:

$$Pr[A_1 \cap \dots \cap A_m] = 1 * \frac{n-1}{n} * \dots * \frac{n-(m-1)}{n}$$

Based on the definition of conditional probability there is another important formula.

Theorem

Law of total probability: Let the events $A_1, \dots, A_n$ be pairwise disjoint and $B \subseteq A_1 \cup \dots \cup A_n$. Then we have:

$$Pr[B] = \sum_{i=1}^n Pr[B|A_i] * Pr[A_i]$$

Intuition: Every summand $Pr[B|A_i] * Pr[A_i]$ is equivalent to $Pr[B \cap A_i]$. Since $A_1, \dots, A_n$ are pairwise disjoint and B is a subset of the union all events $A_1, \dots, A_n$ we can see that summing up all summands gives us $Pr[B]$.

Theorem

Bayes' theorem: Let the events $A_1, \dots, A_n$ be pairwise disjoint and $B \subseteq A_1 \cup \dots \cup A_n$. Then we have:

$$Pr[A_i|B] = \frac{Pr[A_i \cap B]}{Pr[B]} = \frac{Pr[B|A_i] * Pr[A_i]}{\sum_{j=1}^n Pr[B|A_j] * Pr[A_j]}$$

Why do we need this? Bayes' theorem allows us to swap the order of condition (see first factor in numerator of last term). As an example we can look at the evaluation of a medical test: We define $P = \text{"Test is positive"}$, $N = \text{"Test is negative"}$, $S = \text{"Sick"}$, and $H = \text{"Healthy"}$. Often it's easy to determine the probability $Pr[P|S]$ and $Pr[P|H]$. Thanks to Bayes we can now calculate the potentially more interesting information for the patient $Pr[S|P]$:

$$Pr[S|P] = \frac{Pr[P|S] * Pr[S]}{Pr[P|S] * Pr[S] + Pr[P|H] * Pr[H]}$$

2.3 Independence

It may be that a condition does not affect the probability of an event, so $Pr[A|B] = Pr[A]$.

Definition

Events A and B are called **independent** $\Leftrightarrow$

- $Pr[A \cap B] = Pr[A] * Pr[B]$ or
- $Pr[A|B] = Pr[A]$ or
- $Pr[B|A] = Pr[B]$

Multiple events $A_1, \dots, A_n$ are called **independent** if

- $\forall I \subseteq \{1, \dots, n\}$ with $I = \{i_1, \dots, i_k\}$ we have:

$$Pr[A_{i_1} \cap \dots \cap A_{i_k}] = Pr[A_{i_1}] * \dots * Pr[A_{i_k}]$$

So for multiple events to be independent all subsets of those events must be independent.

Useful

Let A, B and C be independent events. Then

- $A \cup B$ and C as well as
- $A \cap B$ and C are independent.

2.4 Random Variables

Often we are interested in some properties of an experiment, for instance how many times we flip tails, or what is the sum of results if we roll a dice x times.

Definition

A **random variable** is a function $X : \Omega \rightarrow \mathbb{R}$. Also we introduce a shorter notation:

$$X \leq 5 = \{\omega \in \Omega | X(\omega) \leq 5\}$$

$X \leq 5$ is short for the event that the random variable X takes a value less than or equal to 5.

As an example we throw a coin three times ($\Omega = \{h, t\}^3$, h short for *heads* and t short for *tails*) and define the random variable Y as how often we flipped heads. For instance, $Y(htt) = 1$ and $Y(ttt) = 0$.

Definition

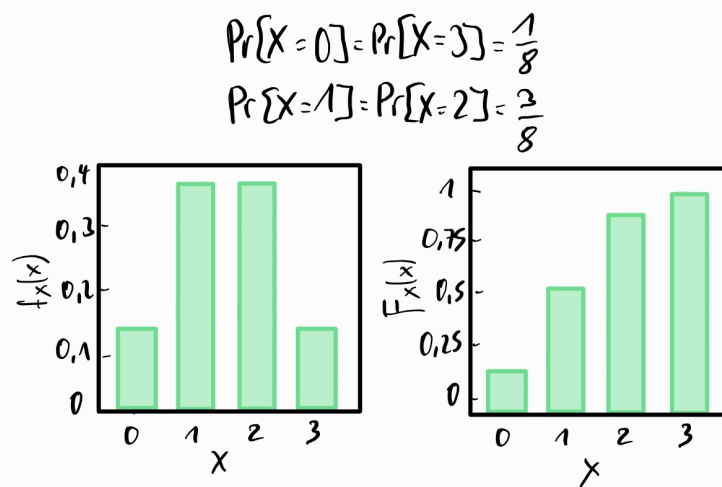
The **probability density function** (PDF) of a random variable X is defined as:

$$f_x : \mathbb{R} \rightarrow [0, 1], x \mapsto \Pr[X = x]$$

We define the **cumulative distribution function** (CDF) of a random variable X as:

$$F_x : \mathbb{R} \rightarrow [0, 1], x \mapsto \Pr[X \leq x] = \sum_{x' \in W_x : x' \leq x} \Pr[X = x']$$

Where W_x is the image of the random variable X (all possible values it can take).

**2.5 Expected Value**

What value can we expect from a given random variable? This is where the expected value comes in.

Definition

Let X be a random variable. We define the **expected value** $\mathbb{E}[X]$ as:

$$\mathbb{E}[X] = \sum_{x \in W_x} x * \Pr[X = x]$$

Or equivalently:

$$\mathbb{E}[X] = \sum_{\omega \in \Omega} X(\omega) * \Pr[\omega]$$

Intuition for the latter definition: We iterate over all elementary events, we plug in the elementary event to the function X (random variable), multiply the result of X by the probability of that elementary event, and sum up all these summands.

In case the image of X is a natural number, we can also use the below definition.

Useful

Let X be a random variable with $W_x \subseteq \mathbb{N}_0$. Then

$$\mathbb{E}[X] = \sum_{i=1}^{\infty} Pr[X \geq i]$$

An important property of the expected value is its linearity.

Useful

For random variables $X_1, \dots, X_n$ and $X = a_1X_1 + \dots + a_nX_n + b$ with $a_1, \dots, a_n, b \in \mathbb{R}$, we have

$$\mathbb{E}[X] = a_1\mathbb{E}[X_1] + \dots + a_n\mathbb{E}[X_n] + b$$

This simple example illustrates the linearity of expected value: Let X be the sum of a dice throw. What is the expected value of the sum of two dice throws?

$$\mathbb{E}[2X] = 2 * \mathbb{E}[X] \tag{6}$$

$$= 2 * (1 * \frac{1}{6} + \dots + 6 * \frac{1}{6}) \tag{7}$$

$$= 2 * (\frac{7}{2}) \tag{8}$$

$$= 7 \tag{9}$$

Definition

For an event $A \subseteq \Omega$ the corresponding **indicator variable** X_A is defined as

$$X_A(\omega) = \begin{cases} 1, & \text{if } \omega \in A \\ 0, & \text{else} \end{cases}$$

The expected value of X_A is $\mathbb{E}[X_A] = Pr[A]$

Useful

$$\top \Rightarrow \mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y] \tag{10}$$

$$\text{if } X, Y \text{ are independent} \Rightarrow \mathbb{E}[X * Y] = \mathbb{E}[X] * \mathbb{E}[Y] \tag{11}$$

2.6 Variance

Variance helps us to find the average deviation of a random variable from its expected value.

Definition

For a random variable X with $\mu = \mathbb{E}[X]$ we define its variance $Var[X]$ as

$$Var[X] = \mathbb{E}[(X - \mu)^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2 = \sum_{x \in W_x} (x - \mu)^2 * Pr[X = x]$$

These three definitions are equal thanks to the linearity property of the expected value.

$$\begin{aligned}
 \mathbb{E}[(X - \mu)^2] &= \mathbb{E}[(X - \mu)^2] \\
 &= \mathbb{E}[X^2 - 2X\mu + \mu^2] && \text{binomial theorem} \\
 &= \mathbb{E}[X^2] - \mathbb{E}[2X\mu] + \mathbb{E}[\mu^2] && \text{linearity of } \mathbb{E} \\
 &= \mathbb{E}[X^2] - 2\mathbb{E}[X]\mu + \mu^2 \\
 &= \mathbb{E}[X^2] - 2\mathbb{E}[X]\mathbb{E}[X] + \mathbb{E}[X]\mathbb{E}[X] && \mu = \mathbb{E}[X] \\
 &= \mathbb{E}[X^2] - \mathbb{E}[X]\mathbb{E}[X] \\
 &= \mathbb{E}[X^2] - \mathbb{E}[X]^2
 \end{aligned}$$

The **standard deviation** of X is $\sigma = \sqrt{\text{Var}[X]}$

Useful

For an arbitrary random variable X and $a, b \in \mathbb{R}$ we have

$$\text{Var}[a * X + b] = a^2 * \text{Var}[X]$$

Useful

$$\begin{aligned}
 \text{if } X, Y \text{ are independent} &\Rightarrow \text{Var}[X + Y] = \text{Var}[X] + \text{Var}[Y] \\
 &\text{Var}[X * Y] \neq \text{Var}[X] * \text{Var}[Y]
 \end{aligned}$$

2.7 Probability Distribution

Often we can group random variables into certain classes (probability distributions).

2.7.1 Bernoulli Distribution

Definition

A random variable X with $W_X = \{0, 1\}$ and density

$$f_X(x) = \begin{cases} p, & \text{for } x = 1 \\ 1 - p, & \text{for } x = 0 \\ 0 & \text{else} \end{cases}$$

is called **Bernoulli distributed**. If $X \sim \text{Bernoulli}(p)$ (X is Bernoulli distributed), then

$$\mathbb{E}[X] = p \text{ and } \text{Var}[X] = p(1 - p)$$

Intuition: *We flip a coin. Does the coin show heads?* Indicator variables are Bernoulli distributed.

2.7.2 Binomial Distribution

If we repeat the experiment of flipping a coin n times and ask ourselves how many times we flipped heads, we can call this outcome binomial distributed.

Definition

A random variable X with $W_X = \{0, 1, \dots, n\}$ and density

$$f_X(x) = \begin{cases} \binom{n}{x} p^x (1-p)^{n-x}, & x \in \{0, 1, \dots, n\} \\ 0, & \text{else} \end{cases}$$

is called **binomial distributed**. If $X \sim \text{Bin}(n, p)$, then

$$\mathbb{E}[X] = n * p \text{ and } \text{Var}[X] = n * p(1-p)$$

Intuition: *We flip a coin n times. How many heads did we get?*

2.7.3 Geometric Distribution**Definition**

A random variable X and density

$$f_X(x) = \begin{cases} p(1-p)^{i-1} & \text{for } i \in \mathbb{N} \\ 0, & \text{else} \end{cases}$$

is called **geometrically distributed**. If $X \sim \text{Geo}(p)$, then

$$\mathbb{E}[X] = \frac{1}{p} \text{ and } \text{Var}[X] = \frac{1-p}{p^2}$$

Intuition: *How often do we need to roll a dice until we get a 6?*

2.7.4 Negative Binomial Distribution**Definition**

A random variable X with density

$$f_X(x) = \begin{cases} \binom{x-1}{n-1} (1-p)^{k-n} p^n & \text{for } k = 1, 2, \dots \\ 0, & \text{else} \end{cases}$$

is called *negative geometrically distributed*. If $X \sim \text{NegBin}(n, p)$, then

$$\mathbb{E}[X] = \frac{n}{p} \text{ and } \text{Var}[X] = \frac{n(1-p)}{p^2}$$

Intuition: *How many times do we need to roll a dice to get n sixes?*

2.7.5 Poisson Distribution

The Poisson distribution focuses on events with very low probability, for instance $X = \text{Amount of heart attacks in Switzerland in the next hour}$. For one person living in Switzerland this probability is very low, however there are about three to four heart attacks across all of Switzerland in one hour.

The Poisson distribution can be interpreted as the limit of a binomial distribution: If we let $X \sim \text{Bin}(n, \frac{\lambda}{n})$ and $n \rightarrow \infty$ then we get the Poisson distribution $X \sim \text{Po}(\lambda)$.

Definition

A random variable X with density

$$f_X(x) = \begin{cases} \frac{e^{-\lambda} \lambda^i}{i!} & \text{for } i \in \mathbb{N} \\ 0, & \text{else} \end{cases}$$

is called *Poisson distributed*. If $X \sim \text{Po}(\lambda)$, then

$$\mathbb{E}[X] = \text{Var}[X] = \lambda$$

2.8 Multiple Random Variables

Often we are interested in multiple random variables simultaneously.

Definition

For two random variables X, Y we call the function

$$f_{X,Y} = Pr[X = x, Y = y]$$

the **joint probability density function** of X and Y .

Using the joint probability density function we can reverse calculate the densities of the individual random variables (marginal distributions).

$$f_X = \sum_{y \in W_Y} F_{X,Y}(x, y) \text{ or } f_Y = \sum_{x \in W_X} F_{X,Y}(x, y)$$

2.8.1 Independence of Random Variables

As in events we can also define independence for random variables.

Definition

Random variables $X_1, \dots, X_n$ are independent $\Leftrightarrow \forall (x_1, \dots, x_n) \in W_{X_1} \times \dots \times W_{X_n}$ we have

$$Pr[X_1 = x_1, \dots, X_n = x_n] = Pr[X_1 = x_1] * \dots * Pr[X_n = x_n]$$

Or alternatively

$$f_{x_1, \dots, x_n}(x_1, \dots, x_n) = f_{X_1}(x_1) * \dots * f_{X_n}(x_n)$$

Useful

Two indicator variables are independent if their events are independent. For two indicator variables we have

$$X \text{ and } Y \text{ are independent} \Leftrightarrow f_{X,Y}(1, 1) = f_X(1) * f_Y(1)$$

Or equivalently

$$Pr[X = 1, Y = 1] = Pr[X = 1] * Pr[Y = 1]$$

Useful

For two independent random variables X and Y we can define $Z = X + Y$ as

$$f_z(z) = \sum_{x \in W_x} f_X(x) * f_Y(z - x)$$

Additionally we have $Bin(n_1, p) + Bin(n_2, p) = Bin(n_1 + n_2, p)$ and $Po(\lambda_1) + Po(\lambda_2) = Po(\lambda_1 + \lambda_2)$.

2.9 Probability Estimations

We want to estimate how likely it is for a random value to differ from its expected value.

Definition

Markov's inequality: Let X be a random variable which only takes non-negative values ($W_X \subseteq \mathbb{N}_0$). Then $\forall t \in \mathbb{R}$ with $t > 0$ we have

$$Pr[X \geq t] \leq \frac{\mathbb{E}[X]}{t}$$

Or equivalently

$$Pr[X \geq t * \mathbb{E}[X]] \leq \frac{1}{t}$$

Markov's inequality comes from ignoring some summands in the definition of the expected value. By applying Markov's inequality to the definition of variance we get Chebyshev's inequality, which can be used for random variables that take negative values.

Definition

Chebyshev's inequality: Let X be a random value and $t \in \mathbb{R}$ with $t > 0$. Then we have

$$Pr[|X - \mathbb{E}[X]| \geq t] \leq \frac{Var[X]}{t^2}$$

Or equivalently

$$Pr[|X - \mathbb{E}[X]| \geq t * \sqrt{Var[X]}] \leq \frac{1}{t^2}$$

And also

$$Pr[X \geq \mathbb{E}[X] + t] \leq \frac{Var[X]}{t^2}$$

As a third estimation we have Chernoff's inequality. It can only be applied on independent Bernoulli distributed random variables (their sums).

Definition

Chernoff's inequality: Let $X_1, \dots, X_n$ be independent Bernoulli distributed random variables with $Pr[X_i = 1] = p_i$ and $Pr[X_i = 0] = 1 - p_i$. Then we have for $X = \sum_{i=1}^n X_i$:

- $Pr[X \geq (1 + \delta)\mathbb{E}[X]] \leq e^{-\frac{1}{3}\delta^2\mathbb{E}[X]}$ for all $0 < \delta \leq 1$
- $Pr[X \leq (1 - \delta)\mathbb{E}[X]] \leq e^{-\frac{1}{2}\delta^2\mathbb{E}[X]}$ for all $0 < \delta \leq 1$
- $Pr[X \geq t] \leq 2^{-t}$

2.10 Coupon Collector Problem

Suppose there are boxes that contain one coupon each. The probability of a certain coupon to be inside a certain box is $\frac{1}{n}$ where n is the number of coupons. We are interested in collecting all coupons. How many boxes do we expect to open in order to collect all coupons?

To solve this problem, we define the random variable X as the number of boxes we need to open until we get all coupons, and X_i as the number of boxes we need to open until we get coupon i . So $X = X_1 + X_2 + \dots + X_n$. X_1 is 1, because we need to open one box to get the first coupon. The probability that we get a new coupon (different to the coupon we just got) is $\frac{n-1}{n}$. Since X_2 is geometrically distributed ("how many times do I need to try until success?"), the expected value for X_2 is $\frac{1}{p} = \frac{1}{\frac{n-1}{n}} = \frac{n}{n-1}$. Analogously $\mathbb{E}[X_3] = \frac{n}{n-2}$ and $\mathbb{E}[X_i] = \frac{n}{n-i+1}$. Then we have:

$$\begin{aligned}
 \mathbb{E}[X] &= \mathbb{E}[X_1 + X_2 + \dots + X_n] \\
 &= \mathbb{E}[X_1] + \mathbb{E}[X_2] + \dots + \mathbb{E}[X_n] && \text{linearity of } \mathbb{E} \\
 &= 1 + \frac{n}{n-1} + \dots + \frac{n}{n-n+1} && \text{as calculated above} \\
 &= \sum_{i=0}^{n-1} n * \frac{1}{n-i} \\
 &= n * \sum_{i=0}^{n-1} \frac{1}{n-i} && \text{move constant } n \text{ out of sum} \\
 &= n * \sum_{i=1}^n \frac{1}{i}
 \end{aligned}$$

As $\frac{1}{n}$ is the harmonic series, we can approximate $\mathbb{E}[X]$ to $n * (\ln(n) + \mathcal{O}(1))$ which is the number of expected boxes we need to open until we collected all coupons.

3 Randomized Algorithms

Up until now we only looked at deterministic algorithms (given a deterministic algorithm $\mathcal{A}$ and input I , then $\mathcal{A}$ will always output $\mathcal{A}(I)$). A randomized algorithm however can make use of randomness. We call the output of such an randomized algorithm $\mathcal{A}(I, \mathcal{R})$, where $\mathcal{R}$ stands for the values the random numbers can take. So generally if we plug in input I to a randomized algorithm and execute it multiple times, we expect different results.

Definition

Monte Carlo algorithms are algorithms that have a fixed run time but we cannot be sure whether the result is correct. **Las Vegas algorithms** on the other hand are algorithms that have an unknown run time but once the algorithm terminates we can be sure that the result is correct.



3.1 Error Reduction

If we execute a randomized algorithms multiple times we can reduce the probability that the algorithm always returns an incorrect result. For instance, let algorithm $\mathcal{A}$ have an error probability of $\frac{1}{3}$ (if we execute $\mathcal{A}$ once we get an error with probability of $\frac{1}{3}$). If we however execute $\mathcal{A}$ 10 times, the probability that all 10 executions are incorrect results is $(\frac{1}{3})^{10} = 1.6935 * 10^{-5}$. We can calculate how often we have to execute an algorithm with success probability $\epsilon > 0$ in order to get an error probability of δ .

Theorem

Error reduction Monte Carlo algorithm: Let $\mathcal{A}$ be a randomized **one-sided error** algorithm with

- $\forall \text{yes-instances: } \Pr[\mathcal{A}(I) = \text{yes}] = 1$ (if input is a *yes*-instance then $\mathcal{A}$ always prints the correct result) and
- $\forall \text{no-instances: } \Pr[\mathcal{A}(I) = \text{no}] \geq \epsilon$ (if input is a *no*-instance then $\mathcal{A}$ prints the correct result with probability greater than ϵ)

then $\forall \delta > 0$ we let $\mathcal{A}_\delta$ be the algorithm that executes $\mathcal{A}$ so many times until $\mathcal{A}$ returns *no* (then $\mathcal{A}_\delta$

also outputs *no*) or until $\mathcal{A}$ has returned *yes* $N = \epsilon^{-1} \ln(\delta^{-1})$ times (then $\mathcal{A}_\delta$ also outputs *yes*). Then we get

$$\Pr[\mathcal{A}_\delta(I) = \text{correct}] \geq 1 - \delta$$

Theorem

Error reduction Las Vegas algorithm: Let $\mathcal{A}$ be a randomized algorithm that never returns an incorrect result but may output an undefined output ??? (??? is printed when we abort the algorithm, for instance if it has been running for too long). Let $\Pr[\mathcal{A}(I) \text{ doesn't print ???}] \geq \epsilon$.

Then $\forall \delta > 0$ we let $\mathcal{A}_\delta$ be the algorithm that executes $\mathcal{A}$ so many times until a value other than ??? is returned (then $\mathcal{A}_\delta$ outputs that value) or until $\mathcal{A}$ has returned ??? $N = \epsilon^{-1} \ln(\delta^{-1})$ times (then $\mathcal{A}_\delta$ also outputs ???). Then we get

$$\Pr[\mathcal{A}_\delta(I) = \text{correct}] \geq 1 - \delta$$

We can also reduce the error probability for two-sided error algorithms (as long as the error probability is $< \frac{1}{2}$).

Theorem

Error reduction two-sided error algorithm: Let $\epsilon > 0$ and $\mathcal{A}$ a randomized algorithm that either prints *yes* or *no*. Let $\Pr[\mathcal{A}(I) \text{ correct}] \geq \frac{1}{2} + \epsilon$.

Then $\forall \delta > 0$ we call $\mathcal{A}_\delta$ the algorithm that calls $\mathcal{A}$ $N = 4\epsilon^{-2} \ln(\delta^{-1})$ times and then prints the result that was received most. Then we have

$$\Pr[\mathcal{A}_\delta(I) \text{ correct}] \geq 1 - \delta$$

Every deterministic algorithm can be seen as a randomized algorithm (with error probability 0). But it is obvious that not every randomized algorithm can be seen as a deterministic one.

3.2 Target Shooting

Let's consider the following problem: *Given all residents of Switzerland, what is the percentage of people having a certain type of cancer?* Or more generally: *Given a set U and a subset $S \subseteq U$ of unknown size, what is $\frac{|S|}{|U|}$?*

We assume that there is a way for us to efficiently determine whether an element of U is also an element of S ($I_S : U \rightarrow \{0, 1\}$). Now we can approximate $\frac{|S|}{|U|}$ by taking a random sample of elements in U and applying I_S to every element in our sample as shown below.

Algorithm 7: target_shooting(U)

- 1 randomly pick $u_1, \dots, u_n \in U$
 - 2 **return** $\frac{1}{n} * \sum_{i=1}^n I_S(u_i)$
-

The next theorem allows us to find the probability that the approximation is within a certain range.

Theorem

Let $\delta, \epsilon > 0$. If $N \geq 3 * \frac{|U|}{|S|} * \epsilon^{-2} * \ln(\frac{2}{\delta})$ then the output of *target_shooting*(U) is with probability of at least $1 - \delta$ in the range of $[(1 - \epsilon) \frac{|S|}{|U|}, (1 + \epsilon) \frac{|S|}{|U|}]$.

3.3 Prime Number Test

We will take a look at a few randomized algorithms that can help us determine whether a number is prime or not.

3.3.1 Euclid Certificate

Algorithm 8: `Euclid_certificate( $n$ )`

```
1 randomly pick  $a \in \{1, \dots, n\}$ 
2 if  $\gcd(a, n) > 1$  then
3   | return false
4 end
5 return true
```

```
1 public static boolean euclid_certificate(int n) {
2     int a = getRandomNumber(1, n);
3     if(gcdByEuclidsAlgorithm(a, n) > 1) {
4         return false;
5     }
6     return true;
7 }
8
9 private static int getRandomNumber(int min, int max) {
10    return (int) ((Math.random() * (max - min)) + min);
11 }
12
13 private static int gcdByEuclidsAlgorithm(int n1, int n2) {
14     if (n2 == 0) {
15         return n1;
16     }
17     return gcdByEuclidsAlgorithm(n2, n1 % n2);
18 }
```

This is a one-sided error algorithm. If the Euclid certificate returns *false* we know that n cannot be prime. If n is not prime then the Euclid certificate prints *true* with probability $\frac{\phi(n)}{n-1} = \frac{|\mathbb{Z}_n^*|}{n-1}$. $\gcd(a, n)$ can be calculated in $\mathcal{O}(\log(an)^3)$ time.

3.3.2 Fermat Certificate

We can make use of Fermat's little theorem.

Theorem

Fermat's little theorem: if $n \in \mathbb{N}$ is prime then $\forall a \in \mathbb{N}$ with $0 < a < n$

$$a^{n-1} \equiv_n 1$$

Algorithm 9: `fermat_certificate( $n$ )`

```
1 randomly pick  $a \in \{1, \dots, n\}$ 
2 if  $\gcd(a, n) > 1$  or  $a^{n-1} \not\equiv_n 1$  then
3   | return false
4 end
5 return true
```

```
1 public static boolean fermat_certificate(int n) {
2     int a = getRandomNumber(1, n);
3     BigInteger Ba = BigInteger.valueOf(a);
4     if(gcdByEuclidsAlgorithm(a, n) > 1 || !(Ba.modPow(BigInteger.valueOf(n-1),
5         BigInteger.valueOf(n)).equals(BigInteger.ONE))) {
6         return false;
7     }
8     return true;
9 }
10 private static int getRandomNumber(int min, int max) {
11     return (int) ((Math.random() * (max - min)) + min);
12 }
13
14 private static int gcdByEuclidsAlgorithm(int n1, int n2) {
15     if (n2 == 0) {
16         return n1;
17     }
18     return gcdByEuclidsAlgorithm(n2, n1 % n2);
19 }
```

The Fermat certificate also has a one-sided error. If it returns *false* we can be sure that n is not prime. But it returns *true* with an error probability of $< \frac{1}{2}$ (assuming n is not a Carmichael number, which is not prime even though it passes the above tests).

3.3.3 Miller-Rabin Certificate

Algorithm 10: miller-rabin_certificate(n)

```
1 if  $n == 2$  then
2 |   return true
3 end
4 if  $n$  is even or  $n == 1$  then
5 |   return false
6 end
7 randomly pick  $a \in \{2, \dots, n-1\}$ 
8 calculate  $k, d \in \mathbb{Z}$  with  $n-1 = d * 2^k$  and  $d$  odd
9  $x = a^d \pmod{n}$ 
10 if  $x == 1$  or  $x == n-1$  then
11 |   return true
12 end
13 while  $i = 0, \dots, k-2$  do
14 |    $x = x^2 \pmod{n}$ 
15 |   if  $x == 1$  then
16 | |   return false
17 |   end
18 |   if  $x == n-1$  then
19 | |   return true
20 |   end
21 end
22 return false
```

```
1 public static boolean miller_rabin_certificate(BigInteger n) {
2     BigInteger one = new BigInteger("1");
3     BigInteger two = new BigInteger("2");
4     if(n.equals(two) || n.equals(new BigInteger("3"))) {
5         return true;
6     }
7     if(n.equals(one) || !odd(n)) {
8         return false;
9     }
10    BigInteger max = n.subtract(one);
11    BigInteger min = new BigInteger("2");
12    BigInteger diff = max.subtract(min);
13    BigInteger a = new BigInteger(max.bitLength(), new Random());
14
15    if(a.compareTo(min) < 0) {
16        a = a.add(min);
17    }
18    if(a.compareTo(diff) >= 0) {
19        a = a.mod(diff).add(min);
20    }
21
22    int k = 0;
23    BigInteger d = n.subtract(one);
24    while(!odd(d)) {
25        k += 1;
26        d = d.divide(two);
27    }
28    BigInteger x = a.modPow(d,n);
29    if(x.equals(one) || x.equals(n.subtract(one))) {
30        return true;
31    }
32    for(int i = 0; i < k-1; i++) {
33        x = x.modPow(two, n);
34        if(x.equals(one)) {
35            return false;
36        }
37        if(x.equals(n.subtract(one))) {
38            return true;
39        }
40    }
41    return false;
42 }
43
44 private static boolean odd(BigInteger o) {
45     BigInteger two = new BigInteger("2");
46     return !o.mod(two).equals(BigInteger.ZERO);
47 }
```

The run time of this algorithm is $\mathcal{O}(\log(n))$. If n is prime then the algorithm always returns *true*. If n is not prime then the algorithm returns *false* with probability $\geq \frac{3}{4}$. As already shown we can minimize the error probability to as much as we like by executing the algorithm multiple times.

3.4 Finding Duplicates

Given a set of data $S = \{s_1, \dots, s_n\}$, how can we find duplicates efficiently (duplicates are $(i, j), 1 \leq i < j \leq n$ such that $s_i = s_j$)? If we are allowed to manipulate the data set we could sort it and then easily find duplicates by traversing through the sorted list. But if there are a lot of elements in S and compare and swap operations are expensive, then we need to find another procedure. We define a hash function $h : U \rightarrow \{1, \dots, m\}$ where U is the universe from which the elements in S come from. We assume that the hash function h can be computed efficiently and that $\forall u \in U, \forall i \in \{1, \dots, m\}$ we have $Pr[h(u) = i] = \frac{1}{m}$ but $s_i = s_j \Rightarrow h(s_i) = h(s_j)$. We hash every element in S and since we let m be much smaller than $|U|$ we have fewer different values and the sort operation is more efficient. But we need to be aware of collisions ($s_i \neq s_j$ but $h(s_i) = h(s_j)$).

3.5 Bloom Filters

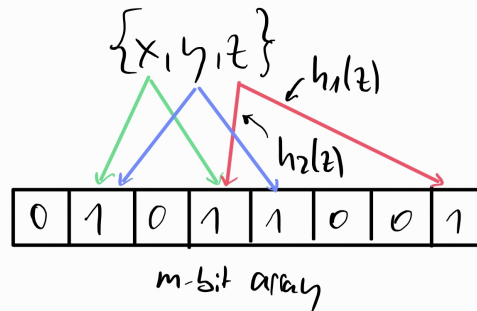
The idea behind bloom filters is that instead of one hash function, we use k different hash functions $h_1, \dots, h_k : U \rightarrow \{1, \dots, m\}$. For every $s_i \in S$ we then have not a single value $h(s_i)$, but a hash vector $(h_1(s_i), \dots, h_k(s_i))$. Moreover, we have a list $\mathcal{L}$ that stores potential duplicates as well as a bit array M with $|M| = m$. Then we proceed using the following algorithm:

Algorithm 11: bloomfilter($S, \{h_1, \dots, h_k\}$)

```

1  $\mathcal{L} = \emptyset$ 
2 for  $s_i \in S$  do
3   for  $h_j \in h_1, \dots, h_k$  do
4     calculate  $x_j = h_j(s_i)$ 
5   end
6   if  $\forall x_j \in \text{hash vector} : M[x_j] = 1$  then
7      $\mathcal{L} = \mathcal{L} \cup \{i\}$ 
8   else
9     Set  $\forall x_j \in \text{hash vector} : \text{set } M[x_j] = 1$ 
10  end
11 end

```



4 Algorithms Highlights

4.1 Networks and Flow

Finding the maximum flow in a network is a problem with many real world applications. First we define a network.

Definition

A **network** is a tuple $N = (V, A, c, s, t)$ with

- (V, A) is a directed graph
- $c : A \rightarrow \mathbb{R}_0^+$ is the capacity function
- $s \in V$ is the source
- $t \in V$ is the target

Intuition: we can imagine a network to be a system of water pipelines where c is the function that specifies how much water can go through a specific pipeline section. Now we can define a flow.

Definition

Let $N = (V, A, c, s, t)$ be a network. A **flow** in N is a function $f : A \rightarrow \mathbb{R}_0^+$ which has to meet the following conditions:

- **capacity constraint:** $\forall e \in A : 0 \leq f(e) \leq c(e)$ (flow on edge cannot be lower than 0 and cannot be higher than capacity)
- **flow conservation:** $\forall v \in V \setminus \{s, t\}$ we have

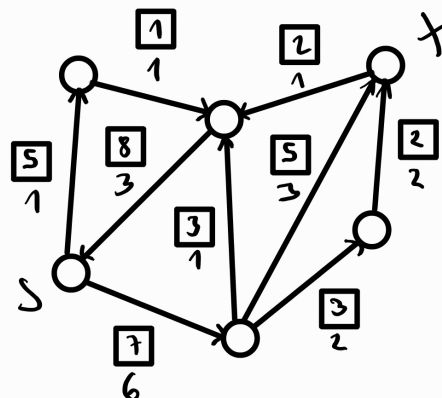
$$\sum_{u \in V : (u, v) \in A} f(u, v) = \sum_{u \in V : (v, u) \in A} f(v, u)$$

Or intuitively: whatever flows in to a vertex also flows out of it, no water is lost (except for s and t).

The **value** of a flow f is defined as

$$\text{val}(f) = \text{netoutflow}(s) = \sum_{u \in V : (s, u) \in A} f(s, u) - \sum_{u \in V : (u, s) \in A} f(u, s)$$

Or intuitively: all the water that enters the network in point s minus all the water that leaves the network in s (netoutflow of s).



Useful

Thanks to flow conservation we have

$$\text{netinflow}(t) = \sum_{u \in V: (u,t) \in A} f(u,t) - \sum_{u \in V: (t,u) \in A} f(t,u) = \text{val}(f)$$

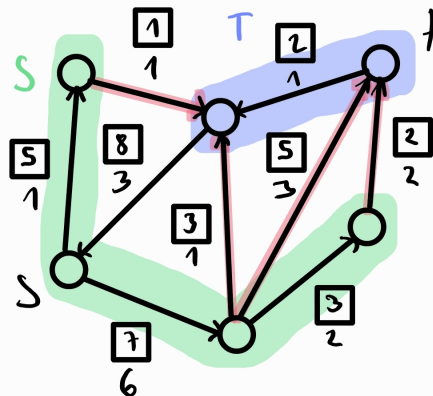
Or intuitively: all the water that reaches t minus all the water that leaves t is equal to the amount of water that entered the network in s .

Definition

An S - T -cut of a network $N = (V, A, c, s, t)$ is a partition (S, T) of V with $S \cup T = V$, $S \cap T = \emptyset$, and $s \in S, t \in T$. The **capacity** of such an S - T -cut is

$$\text{cap}(S, T) = \sum_{(u,v) \in (S \times T) \cap A} c(u, v)$$

Or intuitively: the capacity of all edges that point from partition S to partition T .

**Useful**

Let f be a flow and (S, T) an S - T -cut in $N = (V, A, c, s, t)$. Then we have

$$\text{val}(f) \leq \text{cap}(S, T)$$

The value of a flow cannot be bigger than a S - T -cut, as this would violate the capacity constraint condition.

Theorem

Max-flow min-cut theorem: For every network $N = (V, A, c, s, t)$

$$\max(\text{val}(f)) = \min(\text{cap}(S, T))$$

holds.

4.2 Augmenting Paths

The idea behind many algorithms that try to find the maximum flow in a network is to start from any small flow and to increment it step by step. We now describe the amount by which we increment the flow in each step.

Definition

Let $N = (V, A, c, s, t)$ be a network without backwards pointing edges and let f be a flow in N . The **residual network** $N_f = (V, A_f, r_f, s, t)$ is defined as:

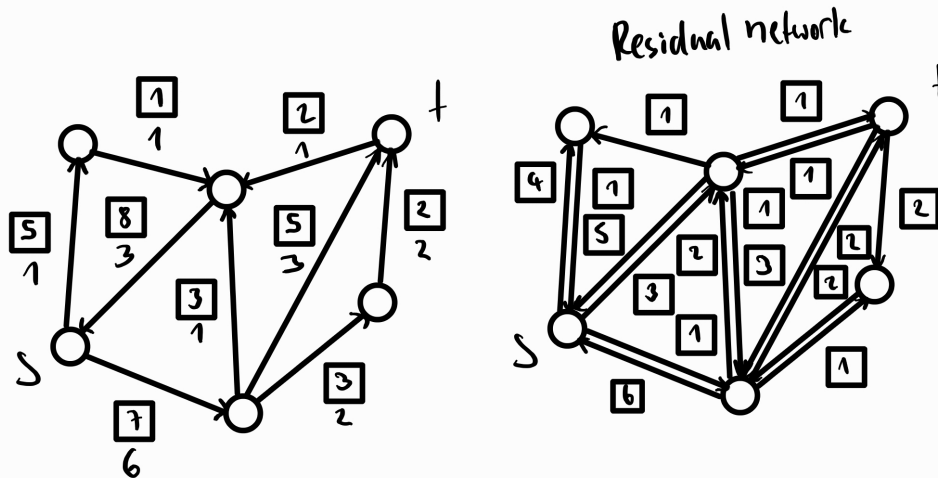
1. If $e \in A$ with $f(e) < c(e)$ then e is an edge in A_f with

$$r_f(e) = c(e) - f(e)$$

2. Is $e \in A$ with $f(e) > 0$ then e^{opp} (direction reversed) is an edge in A_f with

$$r_f(e^{opp}) = f(e)$$

3. A_f only contains edges that obey to rule 1 and 2.



Furthermore we have:

Useful

Let $N = (V, A, c, s, t)$ be a network (without backwards pointing edges) then we have

► a flow f is a maximum flow in $N \Leftrightarrow$ there is no directed s - t -path in the residual network N_f .

Ford-Fulkerson's algorithm lets us find a maximum flow in $N = (V, A, c, s, t)$.

Algorithm 12: ford_fulkerson(V, A, c, s, t)

```

1  $f = 0$ 
2 while  $\exists s - t$ -path  $p$  in  $N_f$  do
3   | find augmenting path and increment flow
4 end
5 return  $f$ 

```

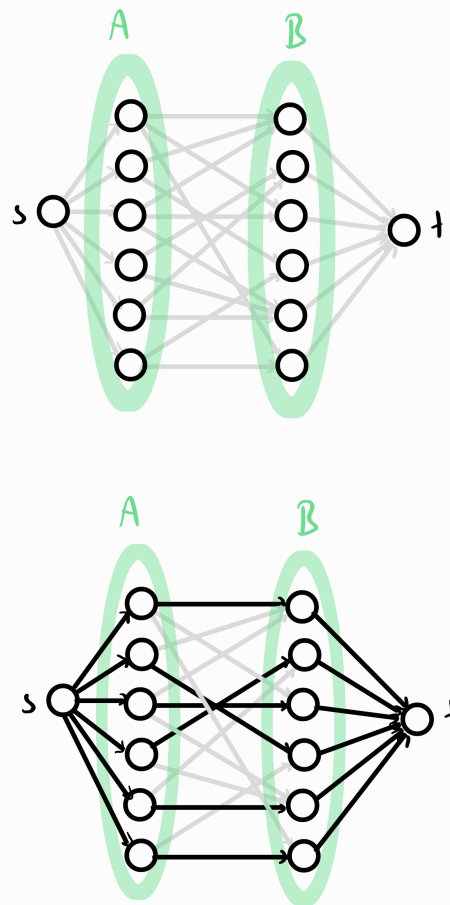
We cannot guarantee that Ford-Fulkerson's algorithm terminates - if the capacities are elements of $\mathbb{R}^+ \setminus \mathbb{N}$ then the algorithm might run forever. We can upper bound the amount of augmentation steps the algorithm performs by $(n - 1) * \max_{e \in Ac}(e)$ because every augmentation increases the flow by at least one so we can have at most $val(f)$ many augmentation steps ($val(f) \leq cap(s, V \setminus \{s\}) \leq (n - 1) * \max_{e \in Ac}(e)$). One augmentation step takes $\mathcal{O}(m)$ time (BFS takes $\mathcal{O}(n + m)$ time but in networks m usually dominates which probably is the reason for the run time $\mathcal{O}(m)$).

Useful

If all capacities in a network are elements of $\mathbb{N}$ and have a value of at most U , then there is a maximum flow that can be computed in $\mathcal{O}(mnU)$ time.

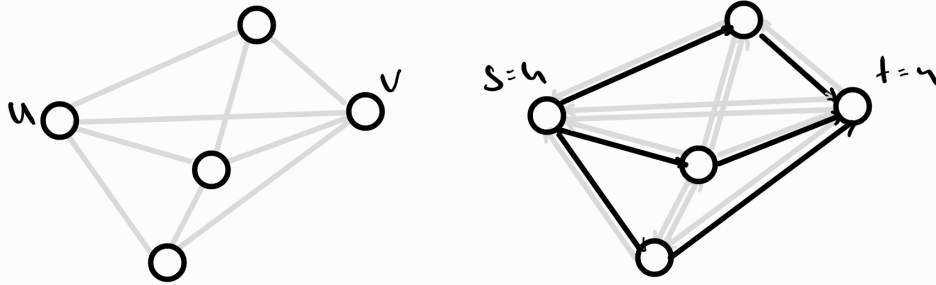
4.2.1 Matchings in Bipartite Graphs

Networks and flows help us to find maximum-cardinality matchings in bipartite graphs ($\max(val(f)) = \max(|M|)$).



4.2.2 Vertex and Edge Disjoint Paths

Let $G = (V, E)$ be a graph. If we want to find the amount of edge disjoint u - v -paths between vertices $u, v \in V, (u \neq v)$ we can model the network as shown below.



Now we execute Ford-Fulkerson's algorithm on G which gives us the maximum flow between u and v which is equivalent to the amount of edge disjoint u - v -paths. We can identify these u - v -paths by finding (and removing or marking as read) a u - v -path and repeating this process $\max(\text{val}(f))$ times.

If we want to calculate vertex disjoint u - v -paths we can replace each vertex $a \in V \setminus \{u, v\}$ by two vertices a_{in} and a_{out} . All edges pointing to a now point to a_{in} and all edges leaving a now leave from a_{out} . a_{in} and a_{out} are connected by an edge with capacity 1. Calculating $\max(\text{val}(f))$ with $s = u, t = v$ gives us the amount of vertex disjoint u - v -paths.

Useful

Ideas for flow problems

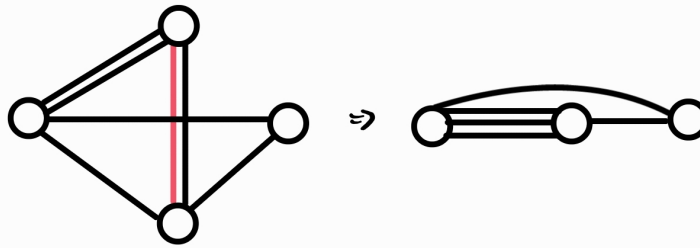
- If we consider a network with undirected edges, we need to draw two edges $\forall u, v \in V, (u \neq v) : (u, v)$ and (v, u) .
- If we want to limit the capacity a certain vertex x is able to take, then we need to replace x by x_{in} and x_{out} and connect them by an edge with desired capacity.
- If we want to simulate multiple sources or multiple targets, we can introduce super-sources or super-targets which are connected to all the sources or targets that we desire to model.

4.3 Minimal Cuts in Graphs

In this chapter we take a look at undirected multigraphs (multiple edges between vertices are allowed here). We are interested in the smallest number of edges we have to remove from the graph $G = (V, E)$, such that the graph is not connected anymore. We call this number $\mu(G)$.

$$\mu(G) = \min_{C \subseteq E: (V, E \setminus C) \text{ is not connected}} |C|$$

We know that if G is not connected then $\mu(G) = 0$. Furthermore, $\mu(G) \leq \min_{v \in V} \deg(v)$. We can now introduce the concept of edge contraction. We merge the two vertices u, v that are incident to that edge in order to get one single vertex $x_{u,v}$. This vertex now is incident to the ones that were incident to u and v , but all edges between u and v are now gone (we can ignore self loops).



We denote the graph that was contracted by edge e as G/e . We can use edge contraction to build the following randomized algorithm.

Algorithm 13: $\text{cut}(G)$

```

1  $G' = G$ 
2 while  $|V(G')| > 2$  do
3    $e = \text{random edge in } G'$ 
4    $G' = (G'/e)$ 
5 end
6 return amount of edges between  $v_1$  and  $v_2$  in  $G'$ 

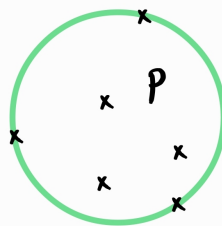
```

The above algorithm picks an edge randomly and contracts the graph on that edge until only two vertices are left. Then it returns the size of the final cut. Since $\mu(G/e) \geq \mu(G)$ the algorithm may output a result that is larger than $\mu(G)$.

4.4 Smallest Enclosing Circle

Let's consider this geometric problem: *Given a finite set of points $P \subset \mathbb{R}^2$, what is the circle C with smallest radius such that all points in P are inside or on C (more formally: $P \subseteq C^\bullet(P)$ where $C^\bullet(P)$ is the set of all points covered by the circle)?*

Smallest enclosing circle C



Useful

For every finite set of points $P \subset \mathbb{R}^2$ there is a **unique** smallest enclosing circle $C(P)$. Additionally, for all P with $|P| \geq 3$ there is a subset $Q \subseteq P$ such that $|Q| = 3$ and $C(Q) = C(P)$.

Now we can build this primitive randomized algorithm.

Algorithm 14: CompleteEnumeration(P)

```

1 for  $\forall Q \subseteq P$  with  $|Q| = 3$  do
2   calculate  $C(Q)$ 
3   if  $P \subseteq C^\bullet(P)$  then
4     return  $C(Q)$ 
5   end
6 end

```

We iterate through all sets $Q \in \binom{P}{3}$, calculate $C(Q)$ in $\mathcal{O}(1)$ and check if $P \subseteq C^\bullet(P)$ in $\mathcal{O}(n)$. Therefore the run time is $\mathcal{O}(n^4)$. We can lower the run time to $\mathcal{O}(n^3)$ with the below algorithm.

Algorithm 15: CleverCompleteEnumeration(P)

```

1  $r = 0$ 
2 for  $\forall Q \subseteq P$  with  $|Q| = 3$  do
3   calculate  $C(Q)$ 
4   if  $r < r(C(Q))$  then
5      $C' = C(Q)$ 
6      $r = r(C(Q))$ 
7   end
8 end
9 return  $C'$ 

```

Instead of checking if $P \subseteq C^\bullet(P)$ we only check if the radius ($r()$ function) of the new circle increased. We can further decrease the run time by "learning" in each round which points are outside of our circle.

Algorithm 16: RandomizedClever(P)

```

1  $P' = P$ 
2 while true do
3   randomly pick  $Q \subseteq P'$  with  $|Q| = 11$ 
4   calculate  $C(Q)$ 
5   if  $P \subseteq C^\bullet(P)$  then
6     return  $C(Q)$ 
7   end
8    $P' = P'$  with all points outside of  $C(Q)$  doubled
9 end

```

If we assume that we can pick Q in $\mathcal{O}(n)$ time then we can execute every round in $\mathcal{O}(n)$ time. Using the sampling lemma we can show that the run time of *RandomizedClever* is $\mathcal{O}(n \log(n))$. This algorithm can also be extended for multiple dimensions and there exist faster algorithms for solving the *Smallest Enclosing Circle* problem.

4.5 Convex Hull

Definition

► **Line segment:** For $v_0, v_1 \in \mathbb{R}^d$ we call

$$\overline{v_0, v_1} = \{(1 - \lambda)v_0 + \lambda v_1 \mid \lambda \in \mathbb{R}, 0 \leq \lambda \leq 1\}$$

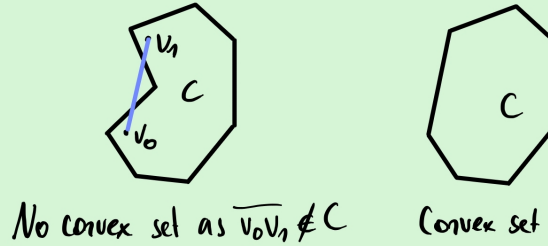
the line segment that connects v_0 with v_1 .

► **Convex set:** A set $C \subseteq \mathbb{R}^d$ is called convex if

$$\forall v_0, v_1 \in C : \overline{v_0, v_1} \in C$$

► **Convex hull:** We define convex hull $\text{conv}(S)$ as the intersection of all convex sets that contain the set $S \in \mathbb{R}^d$.

$$\bigcap_{S \in C \in \mathbb{R}^d, C \text{ convex}} C$$



For a finite set of points in the plane, the convex hull is a polygon. We are now interested in computing the h vertices of that polygon $(q_0, q_1, \dots, q_{h-1})$ (counter clockwise). For simplicity reasons we restrict ourselves to the plane ($S \in \mathbb{R}^2$) and further assume the following:

- No three points lie on a line
- No two points have the same X coordinate

These restrictions are not necessary but make it easier to grasp the concept. Let $S \in \mathbb{R}^2$ be a set of points and $P = \text{conv}(S)$ the convex hull of S . Then we call the pair of points $qr \in P^2, (q \neq r)$ an edge of the convex hull if all points in $P \setminus \{q, r\}$ (and therefore all the points in $S \setminus \{q, r\}$) are left to the directed line $\overrightarrow{q, r}$. This leads us to a more formal definition of the polygon.

Definition

$(q_0, \dots, q_{h-1})$ is the tuple containing the vertices of the polygon P that encloses $\text{conv}(S)$ in counter clockwise fashion if $\forall i \in \{1, \dots, h\} : (q_{i-1}, q_{i \bmod h})$ is an edge of P .

How can we determine if a point lies left to a line?

Useful

Let $p, q, r \in \mathbb{R}^2, (q \neq r)$. p lies left to qr if and only if

$$\det(p, q, r) = \det \begin{pmatrix} p_x & p_y & 1 \\ q_x & q_y & 1 \\ r_x & r_y & 1 \end{pmatrix} > 0 \Leftrightarrow (q_x - p_x) * (r_y - p_y) > (q_y - p_y) * (r_x - p_x).$$

If the two products are equal, then all three points lie on the same line.

Using this we can build an algorithm that finds the polygon $(q_0, \dots, q_{h-1})$. For all $n(n-1)$ pairs qr we check if they are an edge of the polygon (so all remaining $n-2$ points must lie left to qr). This allows us to find the polygon in $\mathcal{O}(n^3)$ time.

4.5.1 Jarvis Wrap

We can do better by using the Jarvis Wrap algorithm.

Algorithm 17: Jarvis_Wrap(S)

```
1 List hull (initiate empty hull)
2  $x =$  point in  $S$  that has the lowest X coordinate (and therefore definitely a vertex of the polygon)
3  $y = x$ 
4 do
5   hull.add( $y$ )
6    $y = \text{find\_next}(y)$ 
7 while  $x \neq y$ ;
8 return hull
```

Algorithm 18: find_next(x)

```
1 pick  $p_0 \in P \setminus \{x\}$  arbitrarily
2  $q_{next} = p_0$ 
3 for  $\forall p \in P \setminus \{x, p_0\}$  do
4   if  $p$  is right to  $xq_{next}$  then
5      $q_{next} = p$ 
6   end
7 end
8 return  $q_{next}$ 
```

```

1 // S contains points in form [[1x,1y],[2x,2y]...,[nx,ny]]
2 public static ArrayList<Integer> jarvis_wrap(int[][] S) {
3     ArrayList<Integer> hull = new ArrayList<Integer>();
4     int x = find_lowest(S);
5     int y = x;
6     do {
7         hull.add(y);
8         y = find_next(y, S);
9     } while(x == y);
10    return hull;
11 }
12
13 private static int find_lowest(S) {
14     int result = 0;
15     int x = Integer.MAX_VALUE;
16     for(int i = 0; i < S.length; i++) {
17         if(S[i][0] < x) {
18             x = S[i][0];
19             result = i;
20         }
21     }
22     return result;
23 }
24
25 private static int find_next(int y, int[][] S) {
26     int q = 0;
27     if(y == 0) {
28         q = 1;
29     }
30     int q_next = q;
31     for(int i = 0; i < S.length; i++) {
32         if(i != y && i != q) {
33             if(!left(i, y, q_next, S)) {
34                 q_next = i;
35             }
36         }
37     }
38     return q_next;
39 }
40
41 private static boolean left(int i, int y, int q_next, int[][] S) {
42     return (S[y][0] - S[i][0])*(S[q_next][1] - S[i][1]) > (S[y][1] - S[i][1])*(S[
43         q_next][0] - S[i][0]);
44 }

```

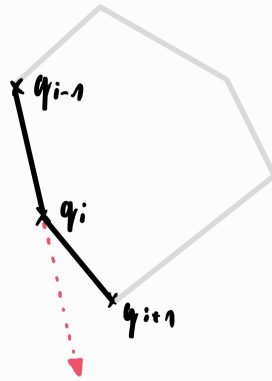
We call *find_next* exactly h times and every call needs $\mathcal{O}(n)$ time.

Useful

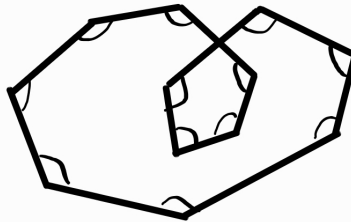
For a set $P \subset \mathbb{R}^2$ of n points, Jarvis Wrap can calculate the convex hull in $\mathcal{O}(nh)$ time where h is the amount of vertices of the convex hull polygon.

4.5.2 Local Repair

Another algorithm to find the convex hull of a set of points in the plane is Local Repair. We know that all points have to be left of an edge of a convex hull polygon. Therefore for all convex hull polygons $(q_0, \dots, q_{h-1})$ it must hold that $\forall i \in \mathbb{N} : q_{i+1 \bmod h}$ is left of $q_{i-1 \bmod h}q_{i \bmod h}$. Intuition:



Alternatively one can also say that the interior angle $\angle q_{i-1} \bmod h q_i \bmod h q_{i+1} \bmod h$ cannot be greater than 180 degrees. But this condition is not strong enough for a valid convex hull as shown in the below counter example.



The idea behind Local Repair is to start with a polygon that is not necessary convex and to 'repair' the polygon successively. By 'to repair' we mean:

Given $(q_0, \dots, q_{k-1})$, if q_i is left of $q_{i-1} \bmod k q_{i+1} \bmod k$, remove q_i from the tuple.

We choose the initial polygon as follows: We sort all points in $S \subset \mathbb{R}^2$ by their X coordinate and get $(s_1, \dots, s_n)$. Now we define the initial polygon as $(s_1, \dots, s_{n-1}, s_n, s_{n-1}, \dots, s_1)$. Below is the pseudo code for Local Repair (its input points must be sorted).

Algorithm 19: Local_Repair($(s_1, \dots, s_n)$)

```
1  $q_0 = s_1$ 
2  $h = 0$ 
3 for  $i = 2, \dots, n$  do
4   while  $h > 0$  and  $q_h$  is left of  $q_{h-1}s_i$  do
5      $h = h - 1$ 
6   end
7    $h = h + 1$ 
8    $q_h = s_i$ 
9 end
10  $h' = h$ 
11 for  $i = n-1, \dots, 1$  do
12   while  $h > h'$  and  $q_h$  is left of  $q_{h-1}s_i$  do
13      $h = h - 1$ 
14   end
15    $h = h + 1$ 
16    $q_h = s_i$ 
17 end
18 return  $(q_0, \dots, q_{h-1})$ 
```

Useful

Given a sequence $(s_1, \dots, s_n)$ of points in $\mathbb{R}^2$ that are sorted by their X coordinate, Local Repair can compute the convex hull of $\{s_1, \dots, s_n\}$ in $\mathcal{O}(n)$ time. Since $\mathcal{O}(n \log n)$ is the optimal time for sorting these points, the total run time of Local Repair (including sorting its input) is $\mathcal{O}(n \log n)$.